

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально-науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 4

з навчальної дисципліни

"Технології розробки корпоративних Web-додатків"

Виконав:

Студент 402 – ТН

Ткаченко Вадим Олександрович

Перевірив:

Тютюнник Петро Богданович

Завдання

Мета роботи — захистити API від несанкціонованого доступу, реалізувати механізм реєстрації та входу користувачів, а також розмежувати права доступу на основі ролей.

1. Реалізація моделі користувача та безпечного зберігання паролів

- Розширте базу даних сутністю `User` (Користувач) та `Role` (Роль), якщо їх ще немає.
- **Вимога:** Паролі користувачів **заборонено** зберігати у відкритому вигляді.
- Реалізуйте механізм хешування паролів перед збереженням у БД (використовуйте алгоритми **BCrypt**, **Argon2** або **PBKDF2**).
- Реалізуйте ендпоінт для реєстрації нового користувача (`Sign Up`), який приймає логін/email та пароль, хешує пароль і зберігає користувача в БД.

2. Реалізація автентифікації (Authentication)

- Реалізуйте механізм входу в систему (`Sign In`).
- Створіть сервіс, який перевіряє валідність наданих облікових даних (порівнює хеш введеного пароля з хешем у БД).
- Оберіть механізм управління сесією. Для REST/GraphQL API рекомендовано використовувати **JWT (JSON Web Tokens)**:
 - При успішному вході сервер має генерувати підписаний токен (`Access Token`), який містить `ID` користувача та його `Role`.
 - Токен має мати обмежений термін дії (наприклад, 15-60 хвилин).

3. Реалізація авторизації (Authorization) та Middleware

- Реалізуйте механізм перехоплення запитів (`Middleware`, `Interceptor` або `Filter`), який перевіряє наявність та валідність токена в заголовках запиту (зазвичай `Authorization: Bearer <token>`).
- Захистіть маршрути (`Endpoints`) вашого API:
 - **Public routes:** Доступні всім (наприклад, реєстрація, логін, перегляд списку товарів).
 - **Authenticated routes:** Доступні тільки авторизованим користувачам (наприклад, створення замовлення, редагування профілю).
 - **Admin routes:** Доступні тільки користувачам з роллю `Admin` (наприклад, видалення товарів, керування користувачами).

4. Інтеграція з поточним API

- Модифікуйте контролери з Лаб №3.
- Спробуйте виконати запит до захищеного ресурсу без токена і переконайтеся, що отримуєте помилку **401 Unauthorized**.
- Спробуйте виконати запит до адміністративного ресурсу зі звичайним користувачем і переконайтеся, що отримуєте помилку **403 Forbidden**.

Очікуваний результат (Звіт та код):

1. Посилання на **Git-репозиторій**.
2. Скріншоти з **Postman/Swagger**:
 - a. Успішна реєстрація (створення користувача).
 - b. Успішний логін (отримання JWT токена у відповідь).
 - c. Знімок БД, де видно, що поле `Password` містить хеш, а не відкритий текст.
 - d. Запит без токена -> статус **401 Unauthorized**.
 - e. Запит з токеном звичайного юзера до адмін-панелі -> статус **403 Forbidden**.
 - f. Успішний запит до захищеного ресурсу з валідним токеном.

Посилання на Git-репозиторій:

https://github.com/tkvadym/TRVD_2026_402TN_Tkachenko_Labs

Хід роботи:

У межах виконання лабораторної роботи було успішно реалізовано підсистему безпеки на базі фреймворку FastAPI. Процес розробки включав декілька ключових етапів, спрямованих на захист даних та контроль доступу.

Першочергово було впроваджено безпечне хешування паролів за допомогою сучасної бібліотеки **bcrypt**. Це гарантує, що конфіденційні дані користувачів зберігаються в базі даних виключно у зашифрованому вигляді, що унеможливорює їх компрометацію навіть у разі отримання доступу до самої БД.

Наступним кроком стало розроблення механізму автентифікації на базі JSON Web Tokens (JWT). Було створено спеціалізований ендпоінт `/auth/login`, який після перевірки облікових даних генерує підписаний цифровим підписом за алгоритмом `HS256` токен. Впроваджений Access-токен має обмежений час життя (одна година) для підвищення рівня безпеки сесій користувачів.

Для покращення користувацького досвіду без втрати рівня безпеки було реалізовано механізм Refresh Tokens. Завдяки ендпоінту `/auth/refresh` користувачі можуть автоматично оновлювати пару токенів перед закінченням дії поточної сесії, що позбавляє їх необхідності повторно вводити логін та пароль.

Окрему увагу було приділено безпеці конфігураційних файлів. Усі чутливі дані, включаючи секретний ключ для підпису токенів, було винесено у змінні оточення сервера через файл `.env`. Це реалізовано за допомогою бібліотеки **python-dotenv**, що запобігає випадковому потраплянню ключів у систему контролю версій.

Також було впроваджено рольову модель доступу (RBAC) через створення спеціальних Middleware-залежностей. Розроблені функції `get_curret_user`, `verify_manager` та `verify_admin` дозволяють здійснювати суворий контроль доступу до різних частин API в залежності від привілеїв користувача. Усі API-контролери, створені на попередніх етапах, були рефакторизовані: ідентифікатор

автора тепер не передається в тілі запиту, а безпечно витягується безпосередньо з JWT-токена.

На завершальному етапі було реалізовано механізм автоматичного наповнення бази даних ("Seeding"). При кожному старті застосунку система перевіряє наявність базових ролей (**User**, **Manager**, **Admin**) та додає їх у разі відсутності. Крім того, виправлено критичну вразливість при реєстрації: тепер поле ролі вилучено з публічної схеми, а всім новим акаунтам за замовчуванням призначається базова роль **User**.

Технічні деталі та контроль доступу

Система підтримує ієрархічну модель розмежування прав, що гарантує цілісність даних та захист ресурсів. Кожен вхідний запит до захищених ресурсів проходить автоматичну перевірку. Якщо токен користувача відсутній або термін його дії закінчився, система повертає стандартну HTTP-помилку `401 Unauthorized`.

У випадках, коли користувач успішно пройшов автентифікацію, але намагається виконати дію, на яку не має прав (наприклад, звичайний користувач намагається видалити чужий проект), сервер повертає статус `403 Forbidden`. При наявності всіх необхідних прав запит виконується успішно зі статусами `200 OK` або `201 Created`.

REST: Swagger UI:

Taskerito API (Lab 4: Auth & JWT) 1.1.0 OAS 3.1

/openapi.json

Authorize

Auth

POST /auth/login Login

POST /auth/refresh Refresh Token

Users

GET /users/ Get All Users

POST /users/ Create User

GET /users/me Get Me

GET /users/{user_id} Get User

PATCH /users/{user_id} Update User

Рєєстрація:

POST /users/ Create User

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "full_name": "Vadym Tkachenko",  "email": "tkachenko@outlook.com",  "password": "mypassword1235"}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  'http://127.0.0.1:8000/users/' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "full_name": "Vadym Tkachenko",  "email": "tkachenko@outlook.com",  "password": "mypassword1235"  }'
```

Request URL

http://127.0.0.1:8000/users/

Server response

Code

Details

201

Response body

```
{  "full_name": "Vadym Tkachenko",  "email": "tkachenko@outlook.com",  "id": 1,  "role_id": 1,  "is_active": true,  "created_at": "2026-03-27T13:49:07.823180"}
```

Response headers

```
content-length: 141  content-type: application/json  date: Fri, 27 Mar 2026 13:49:07 GMT  server: uvicorn
```

Responses

Авторизація (неіснуючий користувач):

Auth

POST /auth/login Login

Parameters

No parameters

Request body required

application/x-www-form-urlencoded

grant_type
string | (string | null)
pattern: "password"

password

☐ Send empty value

username * required
string

asdasdasd

password * required
string(\$password)

scope
string

scope

☒ Send empty value

client_id
string | (string | null)

string

☐ Send empty value

client_secret
string | (string | null)(\$password)

☐ Send empty value

Execute

Clear

Responses

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=asdasdasd&password=asdasdasd&scope=&client_id=string&client_secret=*****'
```

Request URL

http://127.0.0.1:8000/auth/login

Server response

Code	Details
401	Error: Unauthorized

Undocumented

Response body

```
{
  "detail": "Incorrect email or password"
}
```

Download

Response headers

```
content-length: 40
content-type: application/json
date: Fri, 27 Mar 2026 13:26:18 GMT
server: uvicorn
www-authenticate: Bearer
```

Responses

Авторизація:

POST /auth/login Login

Parameters

Cancel

Reset

No parameters

Request body required

application/x-www-form-urlencoded

grant_type

string | (string | null)
pattern: "password\$"

password

☐ Send empty value

username required

string

tkachenko@outlook.com

password required

string(\$password)

scope

string

scope

☒ Send empty value

client_id

string | (string | null)

string

☐ Send empty value

client_secret

string | (string | null)(\$password)

☐ Send empty value

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=tkachenko%40outlook.com&password=myspassword123&scope=&client_id=string&client_secret=*****'
```

Request URL

http://127.0.0.1:8000/auth/login

Server response

Code	Details
200	Response body <pre>{ "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1Iiwicm9sZV9pZCI6ImlwZXhwIjo1Njc5fQ.U9fX76CnsIDXKH3-1PCgrIIVGnuThpZ2hqPK5rG2j7g", "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1Iiwicm9sZV9pZCI6ImlwZXhwIjo1Njc5fQ.LC30eXBLIjoicwVmcwVzaC39.xVjQ8dq0jpX4TVfdKdQwB38v4ma1SPd9RX2W-4solc", "token_type": "bearer" }</pre> Download Response headers <pre>content-length: 352 content-type: application/json date: Fri, 27 Mar 2026 13:27:28 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

Перевірка доступу для не авторизованого користувача:

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /projects/{project_id} Get Project
- Parameters:** project_id (required, integer, path) with value 1.
- Execute:** Button to run the request.
- Responses:** Section showing the result of the request.
- Curl:**

```
curl -X 'GET' \
  'http://127.0.0.1:8000/projects/1' \
  -H 'accept: application/json'
```
- Request URL:** http://127.0.0.1:8000/projects/1
- Server response:**
 - Code:** 401 (Undocumented)
 - Details:** Error: Unauthorized
 - Response body:**

```
{
  "detail": "Not authenticated"
}
```
 - Response headers:**

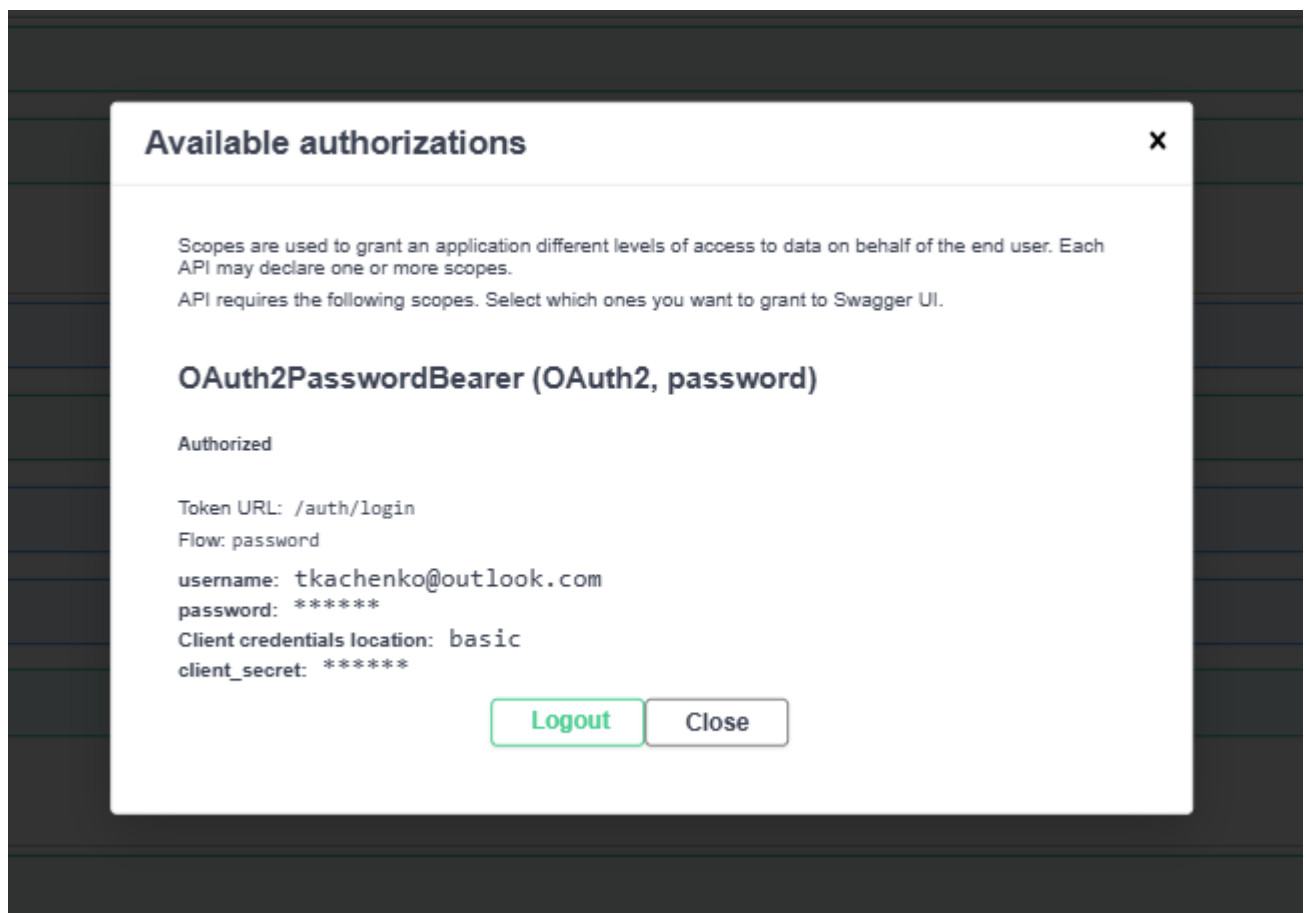
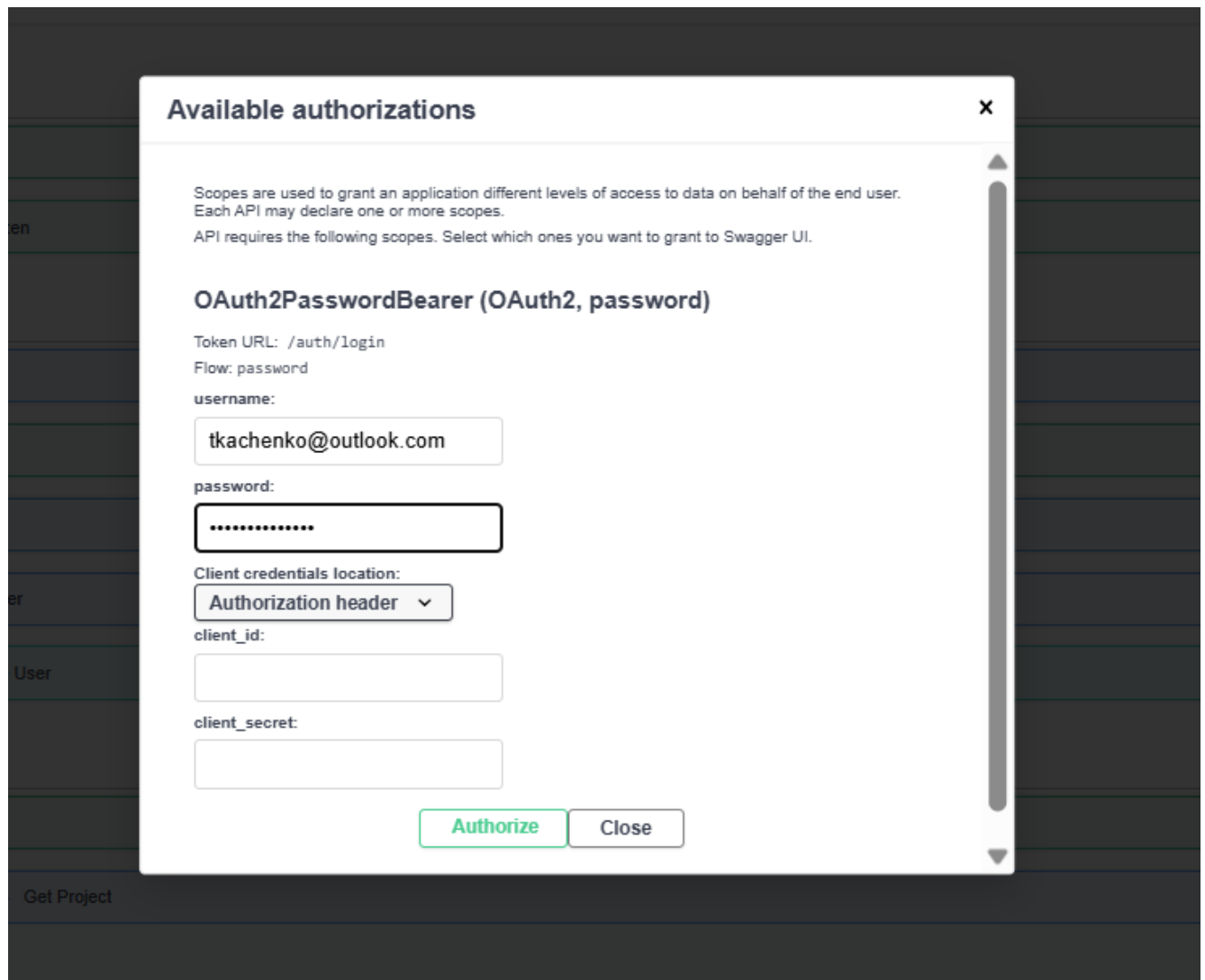
```
content-length: 30
content-type: application/json
date: Fri, 27 Mar 2026 13:29:10 GMT
server: uvicorn
www-authenticate: Bearer
```

Скриншот с БД, де паролі зберігаються у вигляді хешу:

Таблиця: users

	id	full_name	email	password_hash	role_id	is_active	created_at
Ф...	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр
1	1	Vadym Tkachenko	tkachenko@gmail.com	\$2b\$12\$DKSDsr69iugyQcrQXdfV5OY5DLzJ...	1	1	2026-03-27 11:58:37.331287
2	2	Vadym Tkachenko	tkachenko@outlook.com	\$2b\$12\$uNbUX6XQDd2z31RMbVN3..wt6qrUo...	1	1	2026-03-27 13:49:07.823180

Перевірка доступу до захищеного ендпоінта з авторизацією:



GET

/projects/{project_id} Get Project

Parameters

Name	Description
project_id * required	
integer	1
(path)	

Execute

Clear

Responses

Curl

curl -X 'GET' \

'http://127.0.0.1:8000/projects/1' \

-H 'accept: application/json' \

-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ3ZWIiOiIyIiwicm9sZV9pZCI6ImwiZXhwIjoxNzcxOTQwZmQgK1U0w4q8DwPFM6iqJK3osY5__x_SBTIr-L8XGAKbQ'

Request URL

http://127.0.0.1:8000/projects/1

Server response

Code	Details
200	Response body { "name": "Random name", "description": "Another random desc", "status": "Active", "id": 1, "created_by": 1, "created_at": "2026-03-27T12:03:35.692198" } Response headers content-length: 140 content-type: application/json date: Fri, 27 Mar 2026 13:32:29 GMT server: uvicorn

Спроба виконати запит який доступний тільки користувачами з правами ADMIN під користувачем USER:

PATCH

/users/{user_id} Update User

Parameters

Cancel

Reset

Name	Description
user_id * required	
integer	1
(path)	

Request body required

application/json

Edit Value | Schema

{

"role_id": 2

}

Execute

Clear

Responses

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8000/users/1' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ2ZmI0IiY1Iiwicm9sZV9pZCI6IjE0IiwiaWF0Ij0iMTY0ODU4fQ.ZFTYUyXumRfoK6gBagP8TdKq1Pqioej5ezy1PI2tXo' \
  -H 'Content-Type: application/json' \
  -d '{
    "role_id": 2
  }'
```

Request URL

http://127.0.0.1:8000/users/1

Server response

Code Details

403 Error: Forbidden

Response body

```
{
  "detail": "Not enough permissions (requires Admin)"
}
```

Response headers

```
content-length: 52
content-type: application/json
date: Fri, 27 Mar 2026 13:51:37 GMT
server: unicorn
```

Спроба виконати той же самий запит, попередньо назначивши необхідні права (ADMIN):

Таблиця: users							
	id	full_name	email	password_hash	role_id	is_active	created_at
	Ф...	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр
1	1	Vadym Tkachenko	tkachenko@gmail.com	\$2b\$12\$DKSDsr69iuqyQcrQXdfV5OY5DLzJ...	1	1	2026-03-27 11:58:37.331287
2	2	Vadym Tkachenko	tkachenko@outlook.com	\$2b\$12\$uNbUX6XQDd2z31RMbVN3..wt6qrUo...	3	1	2026-03-27 13:49:07.823180

PATCH /users/{user_id} Update User

Parameters

Cancel Reset

Name Description

user_id * required
integer (path)

1

Request body required

application/json

Edit Value Schema

```
{
  "role_id": 2
}
```

Execute Clear

Responses

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8000/users/1' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIiIiwicm9sZV9pZCI6IHNwIjoxNzY1OTU0fQ.ZFTVUxumRfoK6gBagP8TdKq1Pqioej5ezy1PT2tXo' \
  -d '{
    "role_id": 2
  }'
```

Request URL

http://127.0.0.1:8000/users/1

Server response

Code Details

200

Response body

```
{
  "full_name": "Vadym Tkachenko",
  "email": "tkachenko@gmail.com",
  "id": 1,
  "role_id": 2,
  "is_active": true,
  "created_at": "2026-03-27T11:58:37.331287"
}
```

Response headers

```
content-length: 139
content-type: application/json
date: Fri, 27 Mar 2026 13:52:58 GMT
server: uvicorn
```

Responses

Обновить данные в выбранной таблице [F5, Ctrl+R]						
id	full_name	email	role_id	is_active	created_at	
1	Vadym Tkachenko	tkachenko@gmail.com	2	1	2026-03-27 11:58:37.331287	
2	Vadym Tkachenko	tkachenko@outlook.com	3	1	2026-03-27 13:49:07.823180	

БУЛА 1, А СТАЛА 2

Оновлення ACCESS TOKEN за допомогою REFRESH TOKEN:

POST /auth/refresh Refresh Token

Parameters

No parameters

Request body **required**

application/json

Edit Value Schema

```
{
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIiIiwicm9sZV9pZCI6IHNwIjoxNzY1OTU0fQ.ZFTVUxumRfoK6gBagP8TdKq1Pqioej5ezy1PT2tXo"
}
```

Execute

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/auth/refresh' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIiIiwicm9sZV9pZCI6IHNwIjoxNzY1OTU0fQ.ZFTVUxumRfoK6gBagP8TdKq1Pqioej5ezy1PT2tXo"
  }'
```

Request URL

http://127.0.0.1:8000/auth/refresh

Server response

Code Details

200

Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIiIiwicm9sZV9pZCI6IHNwIjoxNzY1OTU0fQ.ZFTVUxumRfoK6gBagP8TdKq1Pqioej5ezy1PT2tXo",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIiIiwicm9sZV9pZCI6IHNwIjoxNzY1OTU0fQ.ZFTVUxumRfoK6gBagP8TdKq1Pqioej5ezy1PT2tXo",
  "token_type": "bearer"
}
```

Response headers

```
content-length: 352
content-type: application/json
date: Fri, 27 Mar 2026 13:55:16 GMT
server: uvicorn
```

Responses